

模型在线服务 MOS v1.2 需求说明书

 产品名称为模型在线服务，Model Online Service，简称 MOS

-  是启发，不是限制。
- 是记录，不是条例。
- 懂业务，懂业务，懂业务，我们是第一个客户。
- 遇到问题或者有更好的想法，随时找高现起。

一、背景

期待已久的 1.2 版本的迭代终于到来了。

种子客户百忙之中抽出来了宝贵的时间测试和使用模型在线服务，并提出了一系列问题，这些问题也正是我们模型在线服务走上生产必须要解决的问题或者需要具备的能力。我们需要在 5 月份持续的有阶段性更新和迭代产品，并希望在 6 月底之前完全解决这些问题。通过这次迭代，我们也希望能够得到客户对我们以及 mos 的认可。

- 1.0 版本的内容，请见 [📖 模型在线服务MOS v1.0需求说明书](#)
- 1.1 版本的内容，请见 [📖 模型在线服务MOS v1.1 需求说明书](#)

二、目标

1.项目整体目标

- 整合分散算力：整合分散化、异构化 AI 算力，以满足推理场景的算力需求为主，为客户提供就近的经济算力。AI 算力平台，使客户像使用 CDN 一样使用 AI 算力。
- 开箱即用：客户不必关心资源在具体哪个节点，运行环境如何。最终给客户具备一定基础设施能力（如 DB、队列等）的算力平台。
- 灵活的售卖方式：提供灵活的售卖方式，包括：按大区划分算力供给。可在大区内进行算力调度。按小时的弹性能力和计费。

2.本次迭代

💡 以下所有内容，围绕本次迭代展开。

2.1 目标（要什么）

- 错误信息规范
- 专属资源组（虚拟机模式）
- 公共资源组 GPU 共享模式
- 自定义镜像支持删除
- 边缘节点构建镜像缓存服务（镜像预热）
- 自动伸缩
- 运营功能：随时可查看当前客户的消耗资源情况

先针对 P0 的问题优先处理。

问题描述	优先级	4月24日老板给客户承诺的时间节点，实际完成时间需要早于这个时间
流量怎么控制分配，就是我昨天提的：我们任务是串行，处理完一个（60s左右不等）才能处理下一个，请求等待的问题。	P0	5月27日
部署启动比较慢	P0	5月15日
日志查看很不方便；日志非常重要	P0	临时方案先交付，产品化正常做
还有很重要流量打标，区分灰度流量和正常流量。	P0	5月30日前
镜像上传慢 docker push registry.mos.ksyun.com/2000176498/clothes:latest	P0	5月15日
我们现在每台机器都有一个代理服务，当前机器所有的任务都会经过这个代理服务。那么这个代理服务需要怎么部署，确保网络稳定	P1	推荐方案
端口设置好像不起作用，我通过启动命令传递是可以的	P2	是一个理解问题
有没有API扩缩容？扩缩容过程中流量重新分配的间隔大约多少？	P2	5月22日

2.2 不要什么？

先解客户问题。

三、需求影响范围 & 方案（落地风险评估）

3.1 新功能

问题描述	优先级	原因分析	需求转化	临时方案
------	-----	------	------	------

流量怎么控制分配，就是我昨天提的：我们任务是串行，处理完一个（60s左右不等）才能处理下一个，请求等待的问题。

比如现在我有 3 个服务实例，来了 5 个任务需要分配。这个流量处理策略怎么控制呢？我们希望 3 个服务实例能同时处理 3 个请求，还有 2 个请求能够排队等待（等待策略我们也可以上层服务自己控制）。

P0

现在的流量路由得策略就是轮询。

客户侧单实例服务是否支持队列和限流？

还是需要的是限流的功能？限什么，比如 QPS

客户需要的是通过动态扩缩容的来应对并发处理能力？
扩容的依据是什么？比如 QPS？负载情况？

 Kimi 智能助手

Q 我

我是一名开发者，nginx做代理服务时是否有能力将请求进行排队？即接入一个请求后，待后端服务器处理数量有空闲后才将请求向后转发，否则仅在nginx处排队等候？

Kimi

Nginx 本身默认没有直接的请求排队功能，但可以通过一些配置和扩展模块来实现类似的效果。

1. 使用 Nginx 的队列管理模块（如 nginx-queue）

- 这是一个第三方模块，它可以对请求进行排队。例如，在配置文件中通过 queue 指令来指定队列的大小和行为。当请求到达时，如果后端服务器处理能力已达上限，请求就会被放入队列中等待。
- 安装这个模块后，在配置文件中可以这样配置：

```
location /api {
    queue 100; # 设置队列大小为 100
    proxy_pass http://backend_servers;
}
```

这样，当后端服务器（backend_servers）无法立即处理新请求时，新来的请求就会在 Nginx 处排队等待，直到有空闲的后端服务器可以处理。

- 这种方式可以有效地控制请求的流入速度，避免后端服务器被过多的请求淹没。不过需要注意的是，这个模块需要单独编译安装，并且需要合理设置队列大小等参数，以达到最佳的流量控制效果。

2. 利用 Nginx 的缓冲功能间接实现排队

- Nginx 的缓冲功能可以在一定程度上起到类似排队的作用。当后端服务器响应较慢时，Nginx 可以先将请求的响应数据缓冲起来。
- 在配置文件中可以设置缓冲区相关参数，如缓冲区大小（proxy_buffer_size）、缓冲区数量（proxy_buffers）等。例如：

```
proxy_buffer_size 128k;
proxy_buffers 4 256k;
```

当后端服务器处理请求速度较慢时，Nginx 会先在缓冲区中存储请求数据。这可以看作是一种简单的排队机制，因为后续的请求会等待缓冲区中的数据被处理后再进行转发。不过这种方式主要是针对响应数据的缓冲，对于请求本身的控制相对有限。

3. 结合其他工具，如 Lua 脚本

- 可以使用 Lua 脚本在 Nginx 中实现更灵活的请求排队逻辑。OpenResty 是一个基于 Nginx 与 Lua 开发的高性能 Web 平台，它提供了一系列 Lua API。
- 例如，通过 Lua 脚本可以创建一个队列数据结构，当请求到达时，先判断后端服务器的负载情况。如果负载过高，就将请求加入这个队列。然后定期检查后端服务器的状态，当服务器有空闲资源时，从队列中取出请求并转发。
- 这种方式比较灵活，可以根据实际的需求和业务逻辑来定制复杂的排队规则。但是需要开发者具备一定的 Lua 编程知识，并且要谨慎处理队列的并发访问等问题。

在实际应用中，是否采用请求排队机制以及如何实现，需要根据具体的业务场景、后端服务器的处理能力和性能要求等因素来综合考虑。



扫描二维码 | 和 Kimi 对话

问问题 · 联网搜 · 读文档

服务支持单实例并发数、请求队列数深度、队列超时时间的设置。排队模式和重试模式。

例如创建服务时，可以指定单实例并发数 1，服务请求队列为 10，超时时间为 60s。
现在该服务有 5 个服务实例，当同时到达 16 个请求，则第 6~15 个请求将会在队列排队，第 16 个

			请求直接返回失败。当第 6~15 个请求在等待处理时间超过 60s 后，也会直接返回失败。	
部署启动比较慢	P0	现在是边缘节点去中心镜像仓库走外网去拉镜像，可能会有 2 方面的原因： • 镜像大, 下载慢（16GB 下载了半个小时左右） • 服务加载镜像慢 • 目前北京中经云的 BGP 带宽为 200Mbps，且镜像仓库和中心的服务共用（中心带宽平时维持在 140Mbps）	把 mos 镜像仓库和中心服务的带宽拆开（5 分钟左右下载完 16GB 的模型文件，大概需要 500Mbps 的带宽，80% 的效率） 多个实例同时拉镜像其实也会出现慢的问题。 也可以和客户沟通一下，客户的期望值是多长时间？ 同时尝试在边缘做镜像缓存，这样不用每次都去中心拉。 三层：宿主机 -- 节点 -- 中心	田老师、韩德田、朱岩
日志查看很不方便；日志非常重要	P0	现在的日志功能只是简单的查看，并不支持分析。 客户想收集的是什么日志？是服务的日志（pod）、服务的请求日志（ingress）？ 日志输出方式有 2 种，一种是 stdout，一种是文件。针对 stdout 的方式，理论上我们是可以收集到的。收集到放哪里？ 针对服务的请求日志（ingress），可以采集，目前还没有想好怎么做。	做一个日志收集器，允许用户配置阿里云和金山云的日志服务。 在服务创建时，可以选择把请求日志（ingress）、服务日志（pod）投递到相应的日志收集器里面。 带宽可能会很大，需要考虑计量计费的问题。或者放在内网，放在文件共享服务里面？或者放在一个公共的 sftp 服务里面，按照 namespace/service/serviceinstance 分目录。	临时方案就是用户在自己的服务里面调用阿里云 / 金山云的日志服务 API/SDK，或者客户的镜像里面安装好日志采集 agent 来进行收集。 先临时把客户运行中产生的日志（在 stdout 上的）全部收集到一个虚拟机或者日志系统里。

还有很重要流量打标，区分灰度流量和正常流量。	P0	这个标签是什么？通过什么方式可以获得？ 想要达到一个什么目的？在什么场景下，达到一个什么目的？ 所以需要的是一个流量管理能力？	流量路由 - 服务 - 服务实例 创建流量路由，设置转发策略，通过路径转发至服务。 客户通过访问流量路由域名来访问后端服务。 例如 /stable --> service01 /beta --> service02 支持优先级。 在后端设计的时候，可以支持转发类型、动作类型等，以便于后续扩展。可参考阿里云 ALB 文档	
镜像上传慢	P0	带宽问题		
docker push registry.mos.ksyun.com/2000176498/clothes:latest				
我们现在每台机器都有一个代理服务，当前机器所有的任务都会经过这个代理服务。那么这个代理服务需要怎么部署，确保网络稳定	P1	我们没有权限登录客户的机器。 建议客户统一维护配置和定期更新。例如通过 nginx 来维护配置文件，通过 crontab 来自动拉取和更新配置文件。		
端口设置好像不起作用，我通过启动命令传递是可以的	P2	待和客户确认		
有没有 API 扩缩容？扩缩容过程中流量重新分配的间隔大约多少？	P2	目前不支持，可以后边增加对应的 OpenAPI。 扩缩容过程中流量重新分配的时间间隔取决于服务实例状态启动的时间和服务就绪时间。		

3.2 优化

- 错误信息规范
- 监控页面的日期选择框

- ~~shell命令行的支持（不只是单行的命令）~~
- ~~服务列表，服务名称支持复制~~
- ~~服务名称一列变宽，超长省略；服务方式一列可以调窄~~
- ~~服务post上传大小限制（已临时改为25MB）~~

四、专有名词 & 规划概述 & 竞品调研

1. 专有名词

名词	解释
流量路由	将访问流量根据转发策略分发到模型在线服务的流量分发控制服务。 该流量路由仅支持模型在线服务。
转发策略	用于确定流量路由实例将请求路由到一个或者多个模型在线服务。

2. 规划概述

近期希望可以支持专属资源和弹性伸缩，以及监控数据接入云监控。

支持专属资源组的价值是消费侧目前主要还是以机器为单位的消耗资源，作为供给侧来是减少库存成本。

支持弹性伸缩的价值是最大限度的解放客户双手，同时不会给客户带来额外的成本，当然，客户已经使用专属资源组。

监控数据接入云监控，但凡客户上生产，监控告警是必备的功能。它能够协助客户及时识别问题，并为客户提供稳定性与安全性的保障。

mos 的流量计费

- 计算资源 
- 公共模型 
- 公共镜像 
- 服务管理 
- 服务实例管理 
- 监控 

- 日志✔
- 部署事件✔
- 支持CPU 类型计算资源✔
- sku 及库存管理✔
- 自定义模型✔
- 自定义镜像✔
- 部署时，需要增加端口✔
- 修改实例规格✔
- 业务域名 ksymos.com，测试环境是 ksymos.cn✔
- 1.1 上线之后，项目每个人都需要找一个模型进行部署和并实现演示效果✔
- 1.1 上线以后，面向公司产研提供一款效率应用@wangfenyi✘
- 扩容改为扩缩容✔
- sku 价格直接显示到列表✔
- 模型在线服务有自己的产品类型✔
- 如何规范错误信息？（1. 错误信息规范 2. 统计 4xx 错误信息，翻译，前端做 mapping）
- 多地域部署（服务组）
- 区域的逻辑（比如降低节点裁撤对客户的影响）
- shell 命令的支持（不只是单行的命令）
- 公共模型直接支持部署
- 部署时，能否直接选模型，而不是现在需要导入一下选文件共享服务的目录
- 数据集的管理
- 模型存储服务
- 部署服务时，镜像支持直接输入镜像地址
- 删除服务时，需要验证码确认，即删除保护
- 公共资源组 GPU 共享模式
- 自动弹性伸缩的能力
- **专属资源组（包年包月）**
- 在线调试
- 流量管理
- 支持VPC
- 按量付费（实时）

- spot 抢占实例
- 包年包月
- 闲忙时计费
- 服务及服务实例的监控告警（目前来看是接入云监控）
- **预置大模型推理服务（按 token 付费）**
- 一键封禁服务域名（当客户服务出现违禁内容时，支持一键封禁服务域名）
- 服务 post 上传大小限制（已临时改为 25MB）
- 多端口支持？
- 自定义模型导入支持 http/https 协议
- 概览页面增加资源概览
- 客户希望使用自己的域名如何解决
- 日志收集服务（例如可以统一收集日志服务，发到金山云或者阿里云的日志服务）
- 支持有状态的服务
- 接云监控

3.竞品调研

💡 阿里云的 EAS 仍然是我们的对标产品，大家多学习学习。

阿里云 人工智能平台 PAI 模型在线服务 (MOS)

腾讯云 TI-ONE 训练平台 模型服务 - 在线服务

 [模型在线服务PAI-EAS_机器学习PAI_在线推理服务_大数据-阿里云](#)

 [高性能应用服务HAI_GPU云服务器_腾讯云](#)

 [ModelArts Studio_MaaS_大模型即服务_华为云](#)

 [UCloud优刻得-首家公有云科创板上市公司](#)

 [Serving 模型部署 | OpenBayes 贝式计算](#)

 [PPIO派欧云](#)

 [DataCanvas Alaya NeW](#)

 [SiliconLLM](#)

针对 1.2 版本相关的功能，竞品分析报告如下：

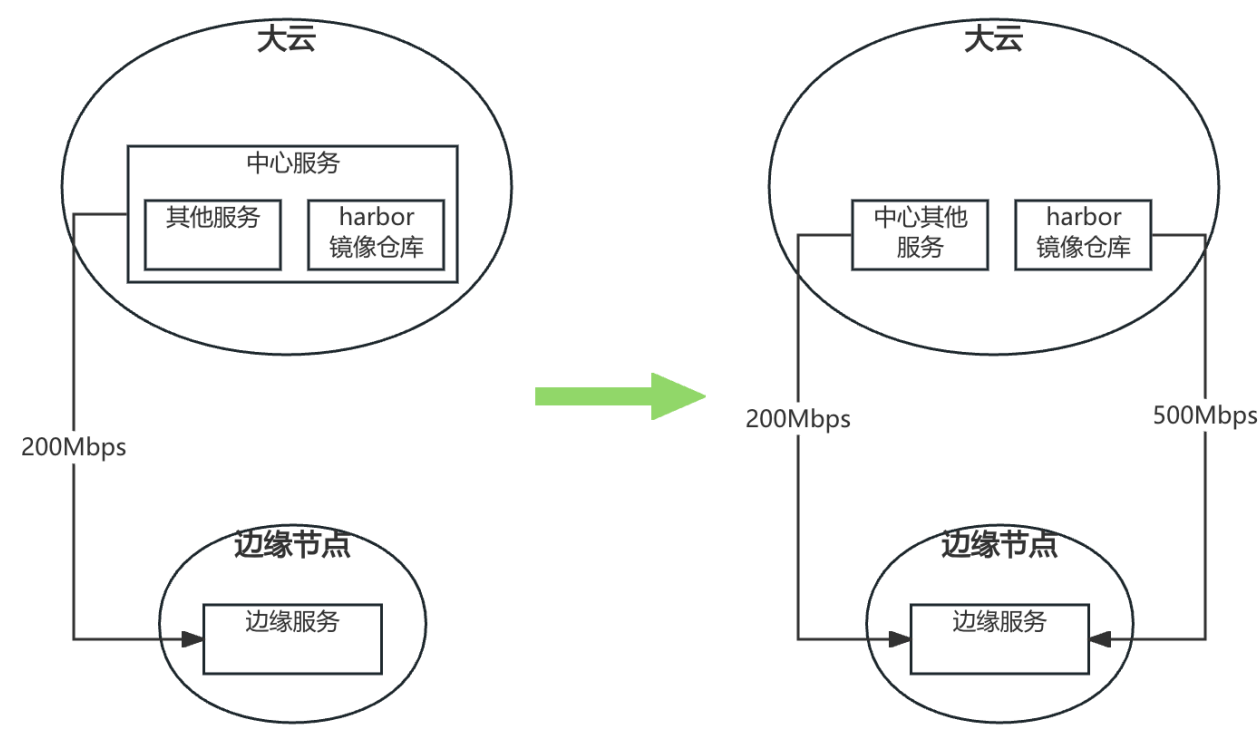
☰ 202505 MOS竞品分析

五、功能需求

1.基础服务需求说明

1.1mos 镜像仓库带宽和中心服务带宽拆开，单独分配 500Mbps 带宽（计费方式和原来的保持一致）

@徐磊



现状：mos 镜像仓库和中心服务共用 200Mbps 带宽，且这 200Mbps 带宽里面，中心服务一般占用带宽在 140Mbps 左右。（16GB 镜像，80% 带宽效率，需要下载 44 分钟左右）

目标：mos 镜像仓库和中心服务带宽拆开，单独分配 500Mbps 带宽。（16GB 镜像，80% 带宽效率，需要下载 5 分钟左右）

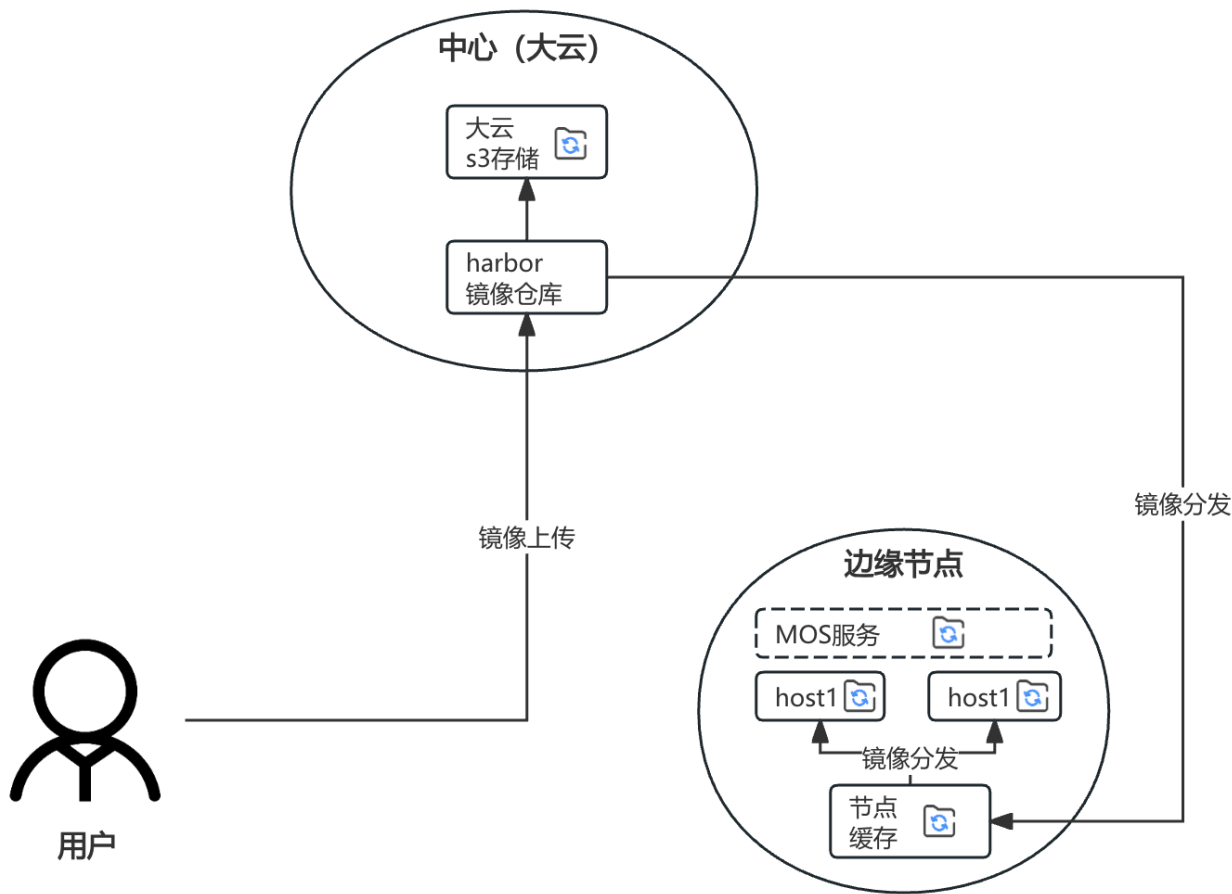
为什么是 500Mbps？我们希望达到的效果是 16GB 大小的镜像可以在 5 分钟左右的时间下载完，按照带宽 80% 的效率计算，我们至少需要 500Mbps 的带宽（ $16GB \times 8 \times 1000 / 5 / 60 / 0.8$ ）。当然，这是理想情况，当多个镜像同时下载时，仍然还是会出现慢的问题。后续可以持续观察带宽消耗情况。

1.2 增加节点缓存，减少回源，减少对带宽的依赖

@贺飞

当镜像上传到 harbor 仓库时，边缘节点会自动同步 harbor 镜像仓库的镜像到节点缓存，再有节点缓存自动分发到宿主机。或者前者自动分发。整体形成大云镜像仓库 --> 节点缓存 --> 宿主机三层的环境结构，不断减少回源，减少对带宽的出网带宽的依赖。

解决 2 个问题：一个是多级缓存。一个是自动分发。



2.控制台需求说明

2.1 交互原型

原型地址：<http://61.130.29.197/#id=m72yt0&p=%E6%A6%82%E8%A7%88>

2.2 新功能

2.2.1 部署服务和更新模型配置支持请求限流功能

@楚得允 @江倩 @李天意

在模型推理服务中，对请求进行限流（如限制单个实例的并发数）的核心意义可总结为两点：

1. 防止资源过载：避免硬件（如 GPU 显存、CPU）被超额请求占满，导致服务崩溃或响应延迟飙升。
2. 保障服务质量：确保每个请求获得稳定资源，维持低延迟和高可靠性，同时公平分配资源，避免少数请求影响整体性能。

——简单来说，限流是用可控的吞吐量换稳定性和可用性。

参数	是否必填	说明	备注
请求限流	选填	可选不限流和单实例最大并发数限制。默认为不限流。 提示信息： 不限流，所有的请求都会直接发给服务实例进行处理。 设置单实例最大并发数时，所有的请求都会进入一个请求队列，按照最大实例并发数的限制向服务实例分配请求，进入队列的请求，超过超时时间后将会返回错误。 选择单实例最大并发数限制时： 单实例最大并发数，默认为 1，范围 1~10000。 超时时间，默认为 60，范围为 1~10000，单位秒。	

2.2.2 增加日志收集的功能

@李雨来 @王率帅 @江倩 @李天意

业务相关的日志，用于对服务状况、服务质量进行分析以及问题排查有着重要意义。理想的做法是做一个日志收集器，用户可以直接配置金山云或者阿里云的日志服务地址，部署服务时，

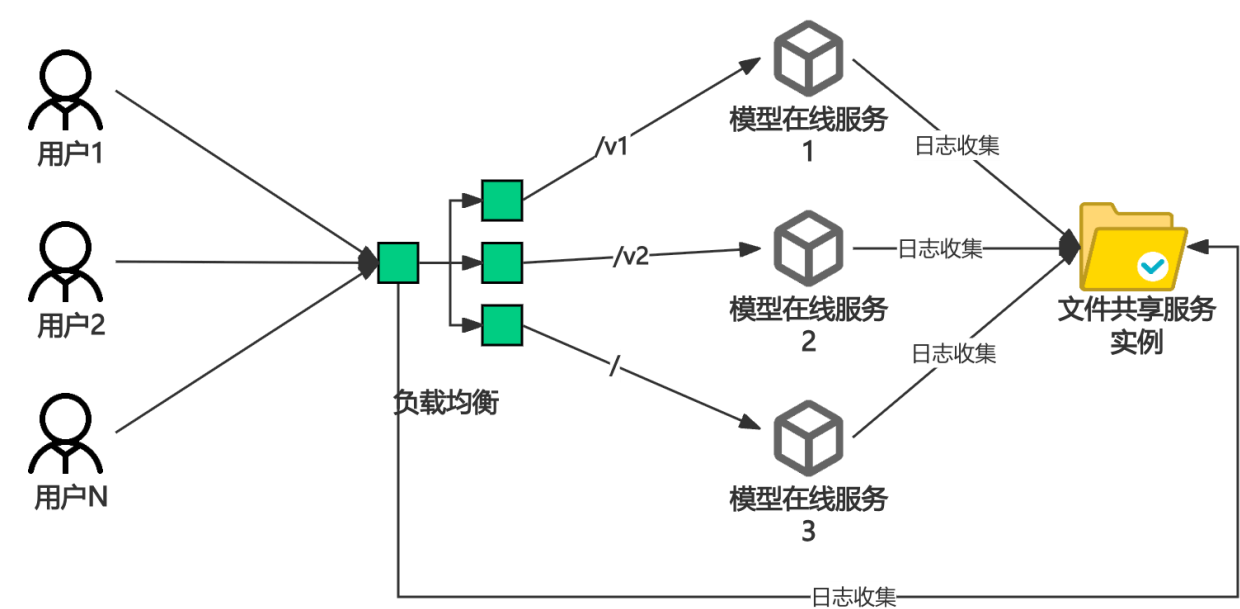
用户可以选择日志收集器，就直接把日志投递到了对应的日志服务平台。

远水接不了近渴，我们先简单的保存为文件，转存到客户的文件共享服务实例里面，用户可以通过多种方式，把改日志搬运到对应的日志服务平台进行分析。

可选方案如下：

方案	方案介绍	优点	缺点
公共 SFTP 服务	把 pod 日志和 ingress 日志直接收集到公共的 SFTP 服务里面，部署一个 SFTPGo 服务，客户需要就直接分一个账号给他。基于 namespaceID 目录给用户授权。	对于我们来说比较简单。	日志存在咱们这，对客户来说有没有数据隐私的顾虑？
专属 SFSS 服务	用户可以打开日志收集的功能，选择把日志收集到自己的文件共享服务里面。		

最终选择专属 SFSS 服务的方案。



参数	是否必填	说明	备注
日志收集	选填	针对节点开通日志收集功能。目前暂只有一个节点。 可选择关闭和开启。默认关闭。 选择开启时，需要选择文件共享服务实例和日志存储路径，而且是必填。 文件共享服务下拉列表的格式和模型那里保持一致即可。 NFS 提示为：请选择文件有我想要的？ 立即前往创建。 日志存储路径，默认为“mos/log/”，允许为空。规则和模型那里路径的规则保持一致即可。（这几个路径的输入框的规则保持一致即可，未来如果需要改的话，一改全改）	pod 日志文件名建议为 “\${instanceid}_\${yyyy-mm-dd}.log” ingress 日志文件名建议为 “\${request}_\${yyyy-mm-dd}.log”

2.2.3 详情增加请求限流的数据展示

@楚得允 @江倩 @李天意

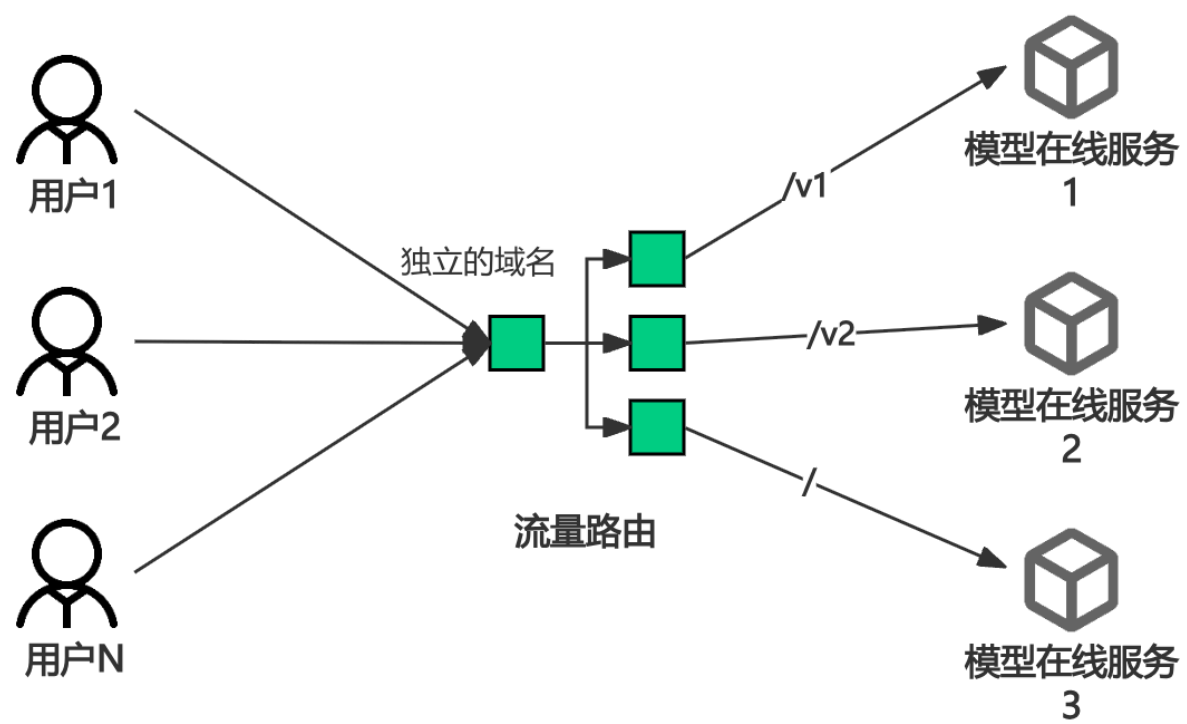
前端和后端同学需额外注意，在详情也需要相应增加这字段数据的展示。

把相应的信息平铺就可以了。label 需要翻译为中文。

2.2.4 流量路由

@刘月 @江倩 @李天意

目前支持基于路径的转发策略，可以帮助业务实现 AB 测试、金丝雀、蓝绿发布等测试场景。



2.2.4.1 新建流量路由

💡 前端文案修改，全局搜索一下，把“地域”改成“节点”。

原因：目前做不到按地域分配资源。现在叫地域有点奇怪，直接和 kenc 保持一致即可。

参数	是否必填	说明	备注
名称	必填	规则同模型服务名称。 提示文案：字母开头，支持数字、小写字母以及中横线 (-)，不得以中横线开始或者结尾，长度位 3~63 位。名称为域名的一部分，全局唯一。 需要判重。	后端需要提供查重的 API 接口。
节点	必填	目前只有厦门电信 41。	
转发策略	必填	每组数据包含了转发规则和目的服务。 转发规则，包含转发类型和路径。 转发类型有 2 种：精确路径匹配、前缀路径匹配。 提示信息： 精确路径匹配，例如设置路径为 /foo，此时只能匹配 /foo，无法匹配 /foo/bar、/foo/baz。 前缀路径匹配，例如设置路径为 /foo，此时可匹配 /foo/、/foo/bar、/foo/baz。 所有转发规则，路径不能重复。路径长度限制为 1~255 个字符，必须以 / 开头，只允许包含字母、数字以及特殊字符 . - _ / = ? : & 目的服务，直接获取该租户下该节点下的服务列表，展示字段包含服务名称、端口、状态，样式为 “\${servcieName}（端口： \${port}/ 状态： \${status}）” 至少有一组数据才能提交。最多支持增加 20 条转发策略。	

2.2.4.2 列表

参数	是否支持排序	是否支持复制	是否支持编辑	说明	备注
名称	支持排序	支持复制		点击名称，进入详情。	
状态	支持排序			枚举值：已生效和未生效。 样式见交互稿。 未生效时，如果错误信息不为空则增加下划线，hover时可以展示错误信息。	
访问域名		支持复制		默认展示前 32 个字符，更多展示 “...”	
节点				枚举值。	
转发策略	支持排序			数字	
创建时间	支持排序			展示样式请见服务列表。	默认按照创建时间排序。最新的在最上边。
修改时间	支持排序			展示样式请见服务列表。	

2.2.4.3 修改

交互同新建，需要数据回显，并将名称和节点置灰。

2.2.4.4 列表搜索

参数	交互	说明
名称	输入框	模糊

2.2.4.5 删除

是否支持批量	不支持批量	
准入条件	任何	
弹窗标题	删除	
操作对象的资源类型	流量路由	
具体操作	删除	
风险提示信息	删除后，访问地址将不可访问，请谨慎操作。	
风险级别	error	常用为 warning、error
针对本次操作，相关的对象信息	名称、节点、转发策略	

2.2.4.6 详情

分类	参数	是否支持复制	支持编辑	说明
基础信息	名称	支持复制		
	访问域名	支持复制		
	节点			枚举值。
	网络计费方式			目前只有按流量付费。
	创建时间			正常展示即可
	修改时间			正常展示即可
转发策略	转发条件			目前仅支持路径
	条件			
	转发动作			目前仅支持转发
	目的服务			

2.3 优化

2.3.1 错误信息规范

错误信息规范的研发工作之前已经做完了，在这个版本合进去就可以了。另外，还有一些前端的工作需要做，这个后边再详细对。

 [MOS错误信息框架](#)

3.API 需求说明

3.1 控制台 API

需要。

3.2 OpenAPI

暂无。

4.监控需求说明

暂无。

5.计费需求说明

暂无。

6.运营需求说明

6.1 USS 系统计费项增加

暂无

6.2 出账单

暂无

六、资源信息

7.1 测试资源

限流功能测试方法

Bash

```
1 算力类型：CPU
2 算力规格：MOS.C1IN.2C4G
3 镜像：library/flaskapp
4 镜像版本：v0.0.1
5 运行命令：python app.py -d 3
6 端口号：5000
7 限流并发数：2
8 限流超时时间：5
9 实例数量：1
10
11 部署完成后：请求：time seq 10 | xargs -P 10 -I {} curl -so /dev/null -w "状态码:%{http_code} cost:%{time_total}\n" http://moscdy.ksymos.cn/a, 域名moscdy.ksymos.cn为你部署的服务名+.ksymos.cn。
12 应该能成功4个，其它的请求会返回503。
```

7.2 生产资源

七、排期上线计划

尽可能列举出所有该需求相关的任务、明细、负责人、截止时间。后可同步至  [边缘计算事务管理](#)

目标：5 月份分阶段上线。

项目阶段	节点	负责人	启动时间	完成时间	备注（输出物，文档可更新到该文档或者提供文档链接）
产品需求	产品需求说明书	@高现起		📅 2025年04月26日 星期六	
	PRD 评审	@高现起		📅 2025年04月27日 星期日	
研发分工	研发分工	@姚鑫		📅 2025年04月28日 星期一	
基础服务 （镜像缓存）（不晚于5月15日上线。）	mos 镜像仓库带宽和中心服务带宽拆开，单独分配500Mbps 带宽	@徐磊		📅 2025年05月12日 星期一	
	增加节点缓存，减少回源，减少对带宽的依赖	@贺飞		📅 2025年05月13日 星期二	

请求限流及等待策略（不晚于5月22日上线。）	控制台 API	@楚得允		📅 2025年05月09日 星期五	
	后端	@楚得允		📅 2025年05月15日 星期四	
	前端	@江倩		📅 2025年05月12日 星期一 T（控制台 API）+1 天	
	联调	@江倩		📅 2025年05月16日 星期五 T（后端）+1 天	
	测试	@李天意		📅 2025年05月21日 星期三 T（联调）+3 天	
	技术方案完成时间 （具备可临时手动配置快速交付的能力）	@楚得允		📅 2025年05月09日 星期五	
	上线	@徐磊		📅 2025年05月22日 星期四	

日志收集 (不晚于5月27日上线。)	后端	@李雨来		 2025年05月16日 星期五 实际完成时间为5月9日。	已完成
	控制台 API	@王率帅		 2025年05月14日 星期三	
	前端	@江倩		 2025年05月16日 星期五	
	联调	@江倩 @王率帅	 2025年05月19日 星期一	 2025年05月19日 星期一	
	测试	@李天意	 2025年05月20日 星期二	 2025年05月21日 星期三	
	技术方案完成时间 (具备可临时手动配置快速交付的能力)	@李雨来		 2025年05月16日 星期五	
	上线	@徐磊		 2025年05月22日 星期四	

流量路由 (不晚于5月30日上线。)	控制台 API	@刘月		📅 2025年05月16日 星期五	
	后端	@刘月		📅 2025年05月20日 星期二	
	前端	@江倩	📅 2025年05月16日 星期五	T (控制台 API) +4 天 📅 2025年05月22日 星期四	
	联调	@江倩	📅 2025年05月22日 星期四	T (后端) +2 天 📅 2025年05月26日 星期一	
	测试	@李天意		T (联调) +3 天 📅 2025年05月28日 星期三	
	技术方案完成时间 (具备可临时手动配置快速交付的能力)	@刘月		📅 2025年05月15日 星期四	
	上线	@徐磊		📅 2025年05月29日 星期四	
文档	官方文档 (含用户手册、产品 release note 等)			📅 2025年06月19日 星期四	
	OpenAPI 文档			📅 2025年06月19日 星期四	
	售前 PPT			📅 2025年06月19日 星期四	
	产品功能清单			📅 2025年06月19日 星期四	📋 边缘计算产品功能清单